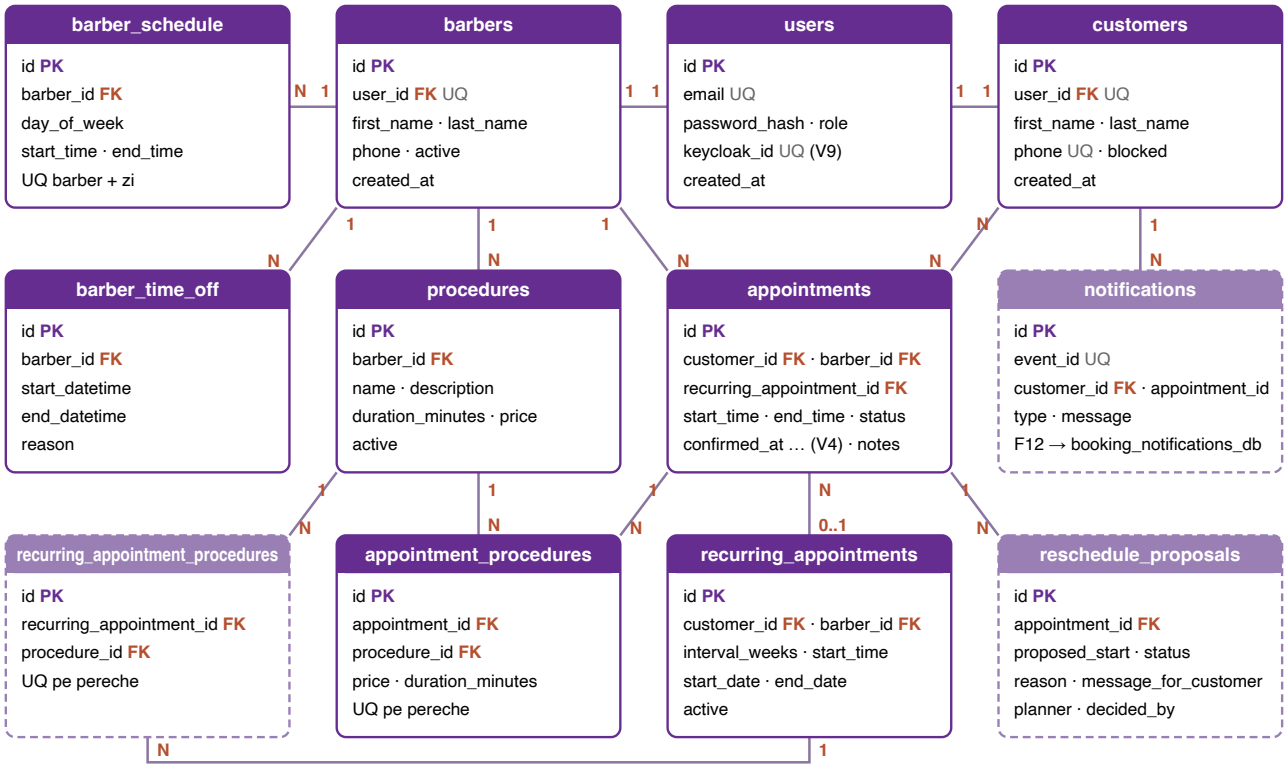
Structura bazei de date

MySQL 8.4 · Flyway · 11 tabele în `booking_api_db` a API-ului + `notifications`, mutată în `booking_notifications_db` la F12 · se construiesc feature cu feature, după `PLAN.pdf` · Rareș

Diagrama



PK cheie primară, `BIGINT` auto-increment · **FK** cheie străină · **UQ** valoare unică · **NN** obligatoriu (`NOT NULL`) · chenar punctat = tabel nou (F4, F8, F11) · eticheta de lângă nume = feature-ul și migrarea care creează sau schimbă tabelul · **coloană cărămizie** = nouă sau schimbată față de codul tău · `recurring_appointments` are și FK spre `customers` și `barbers` (liniile lipsesc din diagramă, ca să nu se încrucișeze)

Tabelele

`users` F2 · V1 · F3 · V2 · F14 · V9

Contul: client (`CUSTOMER`), frizer (`BARBER`), `ADMIN` sau agentul (`AGENT`).

coloană	tip	reguli
`id`	`BIGINT`	PK
`email`	`VARCHAR(150)`	NN, UQ `uk_user_email`
`password_hash`	`VARCHAR(255)`	NN · BCrypt de la F3, `"!"` = de neutilizat (V2) · **dispare în V9**
`keycloak_id`	`VARCHAR(36)`	UQ · `sub` din token · V9
`role`	`VARCHAR(20)`	NN · `CUSTOMER` `BARBER` `ADMIN` + `AGENT` (F3)
`created_at`	`DATETIME`	NN

> Permisiunile rezultă din `role`, dintr-o singură hartă în cod (F3). `ADMIN` vine la pornire din variabilele de mediu (pe EC2 din `.env`, de la F5); `AGENT` îl creează adminul (F8). De la F14 utilizatorul se rezolvă întâi după `keycloak_id = sub`, apoi după email (și se completează `keycloak_id`: legare); abia altfel se creează `users` + `Customer`, doar pentru rolul `CUSTOMER`. Tokenurile `AGENT` nu creează utilizator local. Conturile de frizer și admin se fac în Keycloak cu același email ca profilul local.

`customers` F2 · V1

Profilul clientului: un cont `users` cu rolul `CUSTOMER`.

coloană	tip	reguli
`id`	`BIGINT`	PK
`user_id`	`BIGINT`	NN, UQ, FK → `users.id`
`first_name`	`VARCHAR(100)`	NN
`last_name`	`VARCHAR(100)`	NN
`phone`	`VARCHAR(20)`	NN, UQ `uk_customer_phone`
`blocked`	`BOOLEAN`	NN, implicit `false`
`created_at`	`DATETIME`	NN

> Clientul blocat nu mai rezervă și nu face serii (403); programările lui existente rămân. De la F14, `phone` vine din token (`phone_number`, atribut obligatoriu la înregistrare în realm, pus în token de un mapper), deci rămâne NN.

barbers F2 · V1

Frizerul: un cont `users` cu rolul `BARBER`, creat în aceeași tranzacție (F1).

coloană	tip	reguli
id	BIGINT	PK
user_id	BIGINT	NN, UQ, FK → users.id
first_name	VARCHAR(100)	NN
last_name	VARCHAR(100)	NN
phone	VARCHAR(20)	NN
active	BOOLEAN	NN
created_at	DATETIME	NN

> Frizerul inactiv nu are sloturi; nu se dezactivează cât are programări viitoare ne-anulate (409).

barber_schedule F2 · V1

Programul săptămânal al frizerului: un rând pe zi lucrată.

coloană	tip	reguli
id	BIGINT	PK
barber_id	BIGINT	NN, FK → barbers.id
day_of_week	INT	NN, 1–7
start_time	TIME	NN, < end_time
end_time	TIME	NN

> UQ `uk_barber_day_of_week` pe `(barber_id, day_of_week)` : o zi o singură dată. Programul se înlocuiește total (F1).

barber_time_off F2 · V1

Perioadele în care frizerul nu lucrează.

coloană	tip	reguli
id	BIGINT	PK
barber_id	BIGINT	NN, FK → barbers.id
start_datetime	DATETIME	NN
end_datetime	DATETIME	NN, > start_datetime
reason	VARCHAR(200)	

> Peste o programare ne-anulată → 409. Cu `force=true` (F8, `SCHEDULE_OVERRIDE`) se salvează, iar programările suprapuse trec în `NEEDS_RESCHEDULE`.

procedures F2 · V1

Procedurile unui frizer, cu durata și prețul lui. Nu e un catalog comun.

coloană	tip	reguli
id	BIGINT	PK
barber_id	BIGINT	NN, FK → barbers.id
name	VARCHAR(100)	NN
description	VARCHAR(500)	
duration_minutes	INT	NN, > 0
price	DECIMAL(10,2)	NN, ≥ 0
active	BOOLEAN	NN, implicit true

> UQ `uk_barber_id_procedure_name` pe `(barber_id, name)` : nume unic per frizer (409).

appointments F2 · V1 · F4 · V3 · F6 · V4 · F8 · V5

O programare: clientul, frizerul, intervalul și starea ei.

coloană	tip	reguli
id	BIGINT	PK
customer_id	BIGINT	NN, FK → customers.id
barber_id	BIGINT	NN, FK → barbers.id
recurring_appointment_id	BIGINT	FK, gol la programările normale
start_time	DATETIME	NN
end_time	DATETIME	NN, > start_time
occurrence_date	DATE	V3 · doar de la job, apoi neschimbată
status	VARCHAR(30)	NN · BOOKED CONFIRMED CANCELED COMPLETED + NEEDS_RESCHEDULE (V5)
notes	VARCHAR(500)	
created_at	DATETIME	NN
confirmed_at	DATETIME	V4
completed_at	DATETIME	V4
canceled_at	DATETIME	V4
canceled_by_role	VARCHAR(20)	V4

> UQ `uk_appointment_recurring_occurrence` pe `(recurring_appointment_id, occurrence_date)` (V3): jobul de noapte nu dublează, iar o apariție reprogramată sau anulată nu se recreează (`start_time` se poate schimba, `occurrence_date` nu); MySQL acceptă oricâte NULL, deci programările normale nu sunt atinse. Coloanele din V4 le citesc gauge-ul de ocupare, contorul de anulări târzii (F6) și tiparul de vizite (F7).

appointment_procedures F2 · V1

Procedurile unei programări, cu prețul și durata lor.

coloană	tip	reguli
id	BIGINT	PK
appointment_id	BIGINT	NN, FK → appointments.id
procedure_id	BIGINT	NN, FK → procedures.id
price	DECIMAL(10,2)	NN
duration_minutes	INT	NN

> UQ `uk_appointment_procedure` pe `(appointment_id, procedure_id)` : o procedură o singură dată pe programare.

recurring_appointments F2 · V1 · F4 · V3

O serie: același client, același frizer, aceeași oră, din `interval_weeks` în `interval_weeks`.

coloană	tip	reguli
id	BIGINT	PK
customer_id	BIGINT	NN, FK → customers.id
barber_id	BIGINT	NN, FK → barbers.id
interval_weeks	INT	NN, 1–8
start_date	DATE	NN, în viitor · dă ziua săptămânii
end_date	DATE	NN, după start_date, cel mult 6 luni
start_time	TIME	NN
active	BOOLEAN	NN, implicit true

> Azi tabelul există doar ca entitate. În F4 primește procedurile, în `recurring_appointment_procedures` (V3), și jobul care generează programările pe următoarele 28 de zile.

recurring_appointment_procedures F4 · V3

Procedurile unei serii.

coloană	tip	reguli
id	BIGINT	PK
recurring_appointment_id	BIGINT	NN, FK → recurring_appointments.id
procedure_id	BIGINT	NN, FK → procedures.id

UQ pe (recurring_appointment_id, procedure_id). Procedurile sunt ale frizerului seriei (aceleași reguli ca la POST).

reschedule_proposals F8 · V5 · F10 · V6

O oră nouă propusă de agent pentru o programare rămasă fără frizer, și ce a hotărât clientul.

coloană	tip	reguli
id	BIGINT	PK
appointment_id	BIGINT	NN, FK → appointments.id
proposed_start	DATETIME	NN
reason	VARCHAR(500)	
message_for_customer	VARCHAR(500)	textul pentru client
status	VARCHAR(20)	NN · PROPOSED INVALID ACCEPTED REJECTED EXPIRED
rejection_reason	VARCHAR(50)	BARBER_NOT_WORKING SLOT_TAKEN IN_PAST
proposed_by	VARCHAR(150)	contul care a propus
decided_by	VARCHAR(150)	cine a hotărât
decided_at	DATETIME	
created_at	DATETIME	NN
planner	VARCHAR(10)	NN, implicit RULES · RULES LLM · V6
model_latency_ms	INT	V6

O singură propunere PROPOSED per programare: verificată în service (MySQL nu are index parțial). planner și model_latency_ms sunt datele comparației reguli vs. AI (F10).

notifications F11 · V7 · F12 · V8

O notificare pentru client, scrisă din evenimentele Kafka. Se salvează și se loghează, nu se trimite.

coloană	tip	reguli
id	BIGINT	PK
event_id	VARCHAR(36)	NN, UQ
customer_id	BIGINT	NN, FK → customers.id · fără FK de la F12
recipient_email	VARCHAR(150)	NN, index · customerEmail din eveniment
appointment_id	BIGINT	
type	VARCHAR(30)	NN
message	VARCHAR(500)	NN
created_at	DATETIME	NN
read_at	DATETIME	

UQ pe event_id = idempotența consumatorului Kafka: același eveniment de două ori → un rând. /api/notifications/me filtrează după recipient_email = email-ul din token (serviciul din F12 nu are users). În F12 tabela pleacă în booking_notifications_db (V1__notifications.sql în serviciul nou), iar V8 o șterge din API.

Migrările, feature cu feature. ddl-auto: validate verifică la fiecare pornire că tabelele se potrivesc cu entitățile. Deci o migrare intră **în același commit** cu schimbarea entității, iar una care a rulat (local, în teste, pe EC2) nu se mai editează. Testele rulează migrările pe H2 (MODE=MySQL) până la F5, apoi pe MySQL din Testcontainers: SQL fără ENGINE / CHARSET.

migrare	feature	ce face
V1__schema	F2	cele 9 tabele exact cum le cer entitățile la finalul F1 (F1 nu schimbă nicio entitate), dar users.role VARCHAR(20) și appointments.status VARCHAR(30), nu ENUM: nu se copiază din DDL-ul generat de Hibernate sau din dump-ul bazei locale (ar da enum(...), iar AGENT / NEEDS_RESCHEDULE ar pica la INSERT); ddl-auto: validate; datele demo vin din DataSeeder (profilul demo); baza de pe EC2 (F5) pornește goală și primește V1–V3 la primul start
V2__accounts	F3	parolele care nu încep cu $2 devin "!": parolele în clar din F1–F2 și din seeder devin de neutilizat; conturile se refac prin PUT /api/users/{id}/password sau seeder-ul cu encoder. Fără coloane noi: AGENT intră în enum (doar pentru că V1 are VARCHAR)
V3__recurring	F4	recurring_appointment_procedures + occurrence_date și UQ uk_appointment_recurring_occurrence pe appointments, pentru idempotența jobului (oricâte NULL, și pe MySQL, și pe H2)
V4__appointment_lifecycle	F6	confirmed_at, completed_at, canceled_at, canceled_by_role pe appointments
V5__reschedule_proposals	F8	tabelul reschedule_proposals; NEEDS_RESCHEDULE intră în enum (VARCHAR(30) ajunge, doar pentru că V1 are VARCHAR)
V6__proposal_planner	F10	planner și model_latency_ms pe reschedule_proposals
V7__notifications	F11	tabelul notifications, UQ pe event_id, index pe recipient_email
V8__drop_notifications	F12	în API: DROP TABLE notifications (datele sunt de demo). În booking-notifications: V1__notifications.sql, aceeași tabelă, customer_id fără FK (alt serviciu, altă bază)
V9__users_keycloak	F14	password_hash iese, keycloak_id intră (NULL la utilizatorii existenți, completat la primul login, prin email); utilizatorul local rămâne (clientul și frizerul au profil și programări). În același commit ies PUT /api/users/{id}/password și câmpul password din cereri

Fără tabele: booking-agent (F9–F10) n-are bază de date. Vede totul prin MCP și scrie doar propuneri, prin proposeReschedule.